

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web 2

ENTREGA 1A

Inicialización del Proyecto

Proyecto: gestión-académica

Stack: Next.js 16 · Supabase Cloud · shadcn/ui · TypeScript

Fase 1 — Fundación | MVP (Entregas 1-7)

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Crear un proyecto Next.js 16 con TypeScript utilizando create-next-app.
- Comprender la estructura de carpetas generada por Next.js 16 con App Router.
- Crear y configurar un proyecto en Supabase Cloud (base de datos PostgreSQL).
- Configurar las variables de entorno necesarias para conectar Next.js con Supabase.
- Ejecutar el servidor de desarrollo y verificar que todo funcione correctamente.

Prerrequisitos

Antes de comenzar, asegúrate de tener instalado lo siguiente en tu computadora:

Herramienta	Versión mínima	Cómo verificar
Node.js	18.17 o superior	<code>node --version</code>
npm	9.x o superior	<code>npm --version</code>
Editor de código	VS Code (recomendado)	<code>code --version</code>
Navegador	Chrome / Firefox / Edge	—
Cuenta Supabase	Gratuita	supabase.com
Git	2.x o superior	<code>git --version</code>

Tip: Instalación de Node.js

Si no tienes Node.js, descárgalo desde <https://nodejs.org>. Elige la versión LTS (Long Term Support). Al instalar Node.js, npm se instala automáticamente.

Verificar el entorno de desarrollo

Antes de crear el proyecto, verifica que tu entorno esté correctamente configurado. Abre tu terminal (PowerShell en Windows, Terminal en Mac/Linux) y ejecuta los siguientes comandos:

```
Terminal
node --version
# Esperado: v18.17.0 o superior (ej: v22.x.x)

npm --version
# Esperado: 9.x.x o superior

git --version
# Esperado: git version 2.x.x
```

Si alguno de estos comandos falla o muestra una versión inferior a la requerida, **debes actualizar antes de continuar**. Next.js 16 requiere Node.js 18.17 como mínimo.

Crear el proyecto Next.js 16

Navega a la carpeta donde quieres crear tu proyecto y ejecuta el siguiente comando. Este comando utiliza la herramienta oficial de Next.js para generar un proyecto con la configuración recomendada:

Terminal

```
npx create-next-app@latest gestion-academica
```

El asistente te hará varias preguntas. Responde exactamente como se indica a continuación:cd

Pregunta	Respuesta
Would you like to use TypeScript?	Yes
Would you like to use ESLint?	Yes
Would you like to use Tailwind CSS?	Yes
Would you like your code inside a src/ directory?	No
Would you like to use App Router? (recommended)	Yes
Would you like to use Turbopack for next dev?	Yes
Would you like to customize the import alias?	No

Nota sobre Turbopack

En Next.js 16, Turbopack es el bundler por defecto. Es hasta 400% más rápido que Webpack para el servidor de desarrollo. Ya no necesitas agregar `--turbopack` a tus scripts.

Una vez completado, verás un mensaje de éxito. Ingresa a la carpeta del proyecto:

Terminal

```
cd gestion-academica
```

Explorar la estructura generada

Next.js 16 genera la siguiente estructura de carpetas. Es fundamental que entiendas qué hace cada archivo antes de continuar:

Estructura de carpetas

```
gestion-academica/
├── app/
│   ├── favicon.ico          # Ícono de la pestaña del navegador
│   ├── globals.css         # Estilos globales (Tailwind se configura aquí)
│   ├── layout.tsx          # Layout raíz (envuelve TODAS las páginas)
│   └── page.tsx             # Página principal (ruta "/")
├── public/                  # Archivos estáticos (imágenes, SVGs)
├── .eslintrc.json           # Configuración de ESLint
├── .gitignore               # Archivos ignorados por Git
├── next.config.ts           # Configuración de Next.js
└── next-env.d.ts            # Tipos de TypeScript para Next.js
```

└─ package.json	# Dependencias y scripts
└─ postcss.config.mjs	# Configuración de PostCSS (para Tailwind)
└─ tailwind.config.ts	# Configuración de Tailwind CSS
└─ tsconfig.json	# Configuración de TypeScript

Archivos clave explicados

app/layout.tsx — Es el **layout raíz** de tu aplicación. Envuelve todas las páginas. Aquí se definen las fuentes, metadatos (título, descripción) y los elementos comunes como el `<html>` y `<body>`. Cada página se renderiza dentro del `{children}` de este layout.

app/page.tsx — Es la **página principal** que se muestra cuando el usuario visita la ruta `/`. En Next.js, cada archivo `page.tsx` dentro de `app/` representa una ruta automáticamente.

next.config.ts — Archivo de configuración de Next.js. Aquí se definen opciones como dominios permitidos para imágenes, redirecciones, y configuraciones experimentales. Por ahora lo dejaremos mínimo.

package.json — Define las dependencias del proyecto y los scripts de ejecución. Los scripts principales son:

```

package.json – scripts
{
  "scripts": {
    "dev": "next dev",      // Servidor de desarrollo (Turbopack)
    "build": "next build",  // Compilar para producción
    "start": "next start",  // Ejecutar en producción
    "lint": "next lint"     // Verificar calidad del código
  }
}

```

Tip: App Router vs Pages Router

Next.js tiene dos sistemas de rutas: App Router (carpeta `app/`) y Pages Router (carpeta `pages/`). Nosotros usaremos App Router, que es el recomendado desde Next.js 13+ y el estándar en la versión 16.

Crear el proyecto en Supabase Cloud

Supabase es una plataforma Backend-as-a-Service (BaaS) que provee base de datos PostgreSQL, autenticación, almacenamiento de archivos y APIs automáticas. Vamos a crear un proyecto gratuito:

1. Abre tu navegador y ve a <https://supabase.com>
2. Haz clic en **Start your project** e inicia sesión con tu cuenta de GitHub (o créate una).
3. Haz clic en **New Project** y completa los siguientes datos:

Campo	Valor
Project name	gestion-academica
Database password	Una contraseña segura (GUÁRDALA, la necesitarás)

Region	South America (São Paulo) — la más cercana a Bolivia
Plan	Free (incluye 500 MB de BD y 1 GB de storage)

4. Haz clic en **Create new project**. Espera 1-2 minutos mientras Supabase aprovisiona tu base de datos.
5. Una vez creado, ve a **Project Settings > API** (en el menú lateral izquierdo). Aquí encontrarás las credenciales que necesitamos:

Credencial	Dónde encontrarla
Project URL	Sección "Project URL" — formato: https://xxxxx.supabase.co
Publishable Key (anon)	Sección "Project API keys" — clave pública (sb_publishable_...)
Service Role Key	Sección "Project API keys" — clave secreta (NUNCA exponerla al cliente)

Importante: Seguridad de claves

La Service Role Key tiene acceso total a tu base de datos, saltando todas las políticas de seguridad (RLS). NUNCA la expongas en el código del cliente (navegador). Solo se usa en el servidor (API Routes).

Configurar las variables de entorno

Las variables de entorno permiten almacenar credenciales sensibles sin incluirlas directamente en el código fuente. Next.js las lee automáticamente desde un archivo `.env.local`.

Crea el archivo `.env.local` en la raíz de tu proyecto (al mismo nivel que `package.json`):

```

Archivo: .env.local
# =====
# SUPABASE - Variables de Entorno
# =====
# Estas variables conectan tu app Next.js con Supabase Cloud.
# NEXT_PUBLIC_ = accesible en cliente Y servidor
# Sin prefijo = SOLO accesible en el servidor

# URL de tu proyecto Supabase
NEXT_PUBLIC_SUPABASE_URL=https://TU-PROJECT-ID.supabase.co

# Clave pública (publishable/anon) - segura para el navegador
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_TU-CLAVE-AQUI

# Clave secreta (service role) - SOLO servidor
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIs...TU-CLAVE-AQUI

```

Nunca subas este archivo a Git

El archivo `.env.local` ya está incluido en `.gitignore` por defecto. Verifica que la línea `'.env*.local'` exista en tu `.gitignore`. Si subes tus claves a GitHub, cualquiera podrá acceder a tu base de datos.

Entender el prefijo NEXT_PUBLIC_

Next.js usa un sistema de prefijos para controlar dónde están disponibles las variables de entorno:

Variable	Cliente (navegador)	Servidor
NEXT_PUBLIC_SUPABASE_URL	✓ Disponible	✓ Disponible
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY	✓ Disponible	✓ Disponible
SUPABASE_SERVICE_ROLE_KEY	✗ NO disponible	✓ Disponible

Verificar el archivo .gitignore

Abre el archivo .gitignore en la raíz del proyecto y verifica que contenga estas líneas (Next.js las agrega por defecto, pero es buena práctica confirmar):

Archivo: .gitignore (fragmento relevante)

```
# Dependencias
/node_modules

# Archivos de compilación
/.next/
/out/

# Variables de entorno (CRITICO)
.env*.local

# Misceláneos
*.pem
.DS_Store
```

Buena práctica

Antes de tu primer commit, siempre verifica que .env.local esté en .gitignore. Un solo descuido puede exponer tus credenciales de Supabase al público.

Ejecutar el servidor de desarrollo

Es momento de verificar que todo está correctamente configurado. Ejecuta el servidor de desarrollo:

Terminal

```
npm run dev
```

Deberías ver un mensaje similar a:

Salida esperada en terminal

```
▲ Next.js 16.2.x (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.x.x:3000

✓ Ready in XXms
```

Abre tu navegador y visita <http://localhost:3000>. Deberías ver la página de bienvenida de Next.js con el logo de Vercel y enlaces a la documentación.

Nota sobre Turbopack

Observa que la terminal muestra "Turbopack" junto a la versión de Next.js. Esto confirma que el bundler rápido está activo. En versiones anteriores, había que activarlo manualmente con `--turbo`.

Personalizar la página principal

Vamos a reemplazar el contenido por defecto con algo propio para confirmar que podemos editar y ver los cambios en tiempo real (Hot Reload).

Abre `app/page.tsx` y reemplaza TODO su contenido con:

```
Archivo: app/page.tsx
export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center bg-gray-50">
      <div className="text-center space-y-4">
        <h1 className="text-4xl font-bold text-gray-900">
          Gestión Académica
        </h1>
        <p className="text-lg text-gray-600">
          Sistema de gestión universitaria
        </p>
        <p className="text-sm text-gray-400">
          Next.js 16 · Supabase · shadcn/ui
        </p>
      </div>
    </main>
  );
}
```

Guarda el archivo. **Sin reiniciar el servidor**, vuelve al navegador. La página se actualiza automáticamente mostrando "Gestión Académica" centrado en la pantalla.

Fast Refresh

Next.js detecta tus cambios automáticamente y actualiza el navegador sin recargar la página completa. Esto se llama Fast Refresh y conserva el estado de tus componentes React entre ediciones.

Verificación Final

Antes de continuar con la Entrega 1B, confirma que todos estos puntos se cumplen:

- ✓ `node --version` muestra v18.17 o superior
- ✓ El proyecto `gestion-academica` fue creado sin errores
- ✓ El archivo `.env.local` existe con las 3 variables de Supabase

- ✓ `.env.local` está incluido en `.gitignore`
- ✓ `npm run dev` inicia sin errores y muestra Turbopack
- ✓ `http://localhost:3000` muestra "Gestión Académica"
- ✓ Los cambios en `page.tsx` se reflejan automáticamente (Fast Refresh)

Ejercicio Propuesto

Para reforzar lo aprendido, realiza las siguientes tareas por tu cuenta:

6. **Modifica el título:** Cambia "Gestión Académica" por el nombre de tu universidad y verifica que el Fast Refresh funcione.
7. **Explora el layout:** Abre `app/layout.tsx` y cambia el título en `metadata.title` por "Gestión Académica". Recarga la página y observa la pestaña del navegador.
8. **Investiga el App Router:** Crea una carpeta `app/about/` con un archivo `page.tsx` dentro que muestre "Acerca de". Luego visita `http://localhost:3000/about`. ¿Funciona sin configuración adicional? ¿Por qué?
9. **Verifica Supabase:** Ingresa al Dashboard de Supabase, ve a Table Editor y confirma que tu proyecto está activo (verás las tablas del esquema `auth`).

Próximo: Entrega 1B

En la siguiente entrega instalaremos `shadcn/ui`, configuraremos Tailwind CSS, crearemos la estructura completa de carpetas del proyecto, y ejecutaremos el esquema SQL de la base de datos en Supabase Dashboard.